Cyber-Cat: Heterogeneous Air-Ground Pursuit System — Final Report

Team 2: Sujie Zong, Wei Yun Feng, Ruiling Jin, Ching-Hsin Hsu, Zhixuan Fu

1. Project Overview

Cyber-Cat is a heterogeneous air-ground pursuit system in which a TurtleBot 4 autonomously detects and chases a Crazyflie drone using onboard vision, implementing three core behaviours: aerial-target detection, sustained pursuit with standoff regulation, and recovery when the target leaves frame. Pursuit is initiated via keyboard command, and target detection relies on a pink ball affixed to the drone, chosen for its strong colour contrast in the lab environment.

The full pipeline was validated in both Ignition Gazebo simulation and on physical hardware. A proportional visual-servo controller governs pursuit, with following distance maintained through an area-ratio standoff rule. Real-world trials confirmed reliable tracking up to approximately 0.5 m drone altitude, beyond which the marker becomes too small for consistent detection.

2. Technical Specification

2.1 Architecture

The Crazyflie (pink ball affixed) is controlled in simulation via a ROS 2 waypoint node, and on hardware via the cflib Python library communicating directly over a Crazyradio 2.0 USB dongle without ROS 2. The TurtleBot 4 (turtlebot468) captures RGB frames with its OAK-D camera and drives with its Create 3 base, communicating with the laptop over WiFi through ROS 2 Humble topics. All behaviour nodes run on a laptop connected to the TurtleBot over WiFi. In simulation, ros_ign_bridge translates between Ignition Gazebo's internal bus and ROS 2 topics

2.2 Nodes

`vision_processor_node.py` — subscribes to `/turtlebot468/oakd/rgb/preview/image_raw` (sensor-data QoS). Each frame is reshaped into a NumPy array, converted to HSV with a pure-NumPy implementation, and masked for **hue ∈ [140, 170], saturation ≥ 50, value ≥ 80**, which is the pink/magenta band. A minimum of 25 matched pixels is required before declaring a detection. The node publishes `/vision/detection` (`"target_found"` / `"no_target"`) and `/vision/target_info` (normalised centroid `norm_cx`, `area_ratio`, bounding box) at ~10 fps (every third incoming frame). All HSV thresholds are ROS 2 parameters tunable at runtime.

`mission_executive_node.py` — a 10 Hz four-state FSM publishing `/mission/state` and controlling the TurtleBot 4 light ring via `irobot_create_msgs/LightringLeds`.

State	Light	Entry	Exit
IDLE	White	Start / "stop" command	"start" typed → SEARCH
SEARCH	Blue	IDLE or target loss	≥ 6/10 frames detected → CHASE; 90 s timeout → IDLE
CHASE	Yellow	Confirmed detection	`area_ratio > 0.025` held 1.5 s → CAPTURE; lost > 3 s → SEARCH
CAPTURE	Green flash	Close sustained detection	5 s celebration → restart SEARCH if not stopped

Detection flicker is suppressed by a rolling `deque` of length 10: confirmed detection requires sum ≥ 6, confirmed loss requires sum < 2. The capture guard additionally requires simultaneous confirmed detection, preventing false triggers from a single close-range frame.

`chase_controller_node.py` is a 10 Hz controller publishing `geometry_msgs/Twist` to `/turtlebot468/cmd_vel`. The drone's normalised horizontal position in frame (`norm_cx`: 0 = left, 0.5 = centre, 1 = right) drives a proportional angular command capped at ±1.2 rad/s; a ±5% deadzone suppresses jitter. Forward speed peaks when the drone is centred and scales down during hard turns. Proximity is estimated from `area_ratio` (fraction of pixels that are pink), with three forward-speed zones:

Zone	area_ratio	Forward speed
Far	< 0.007	Full (up to 0.4 m/s)
Braking	0.007 to 0.018	Linearly ramps to zero
Standoff	≥ 0.018	Zero

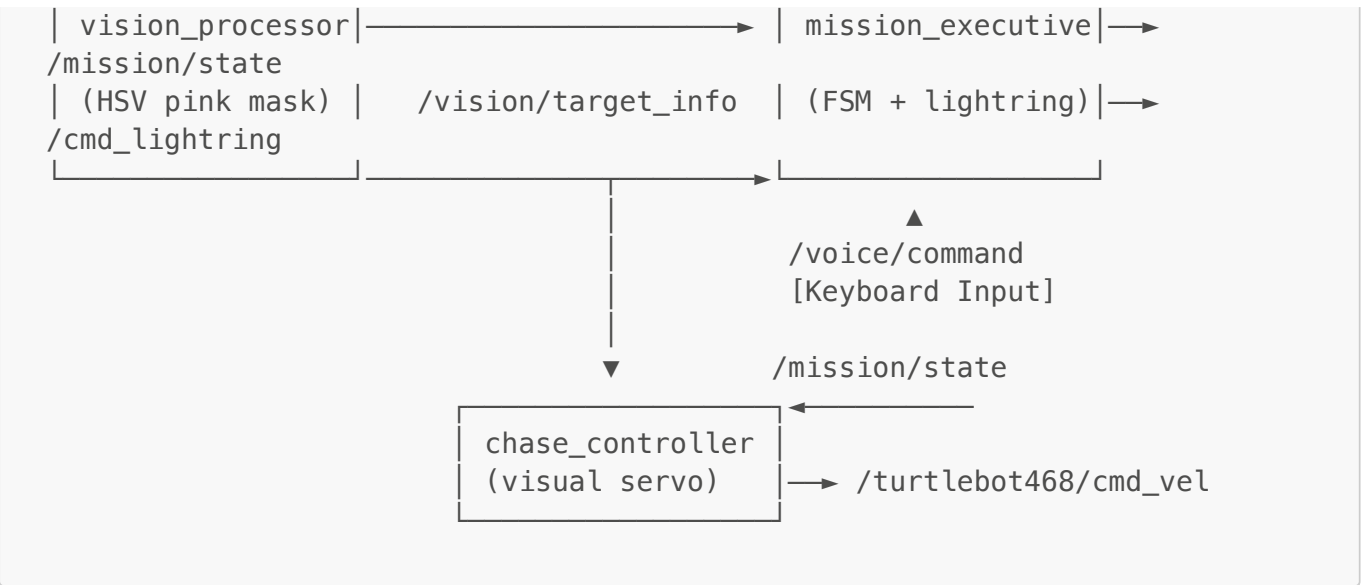
On detection loss, a 1 s grace period turns toward the last known bearing at zero forward speed before reverting to spin-to-reacquire.

`voice_command_node.py` — non-blocking stdin reader at 10 Hz publishing typed commands (`start`, `stop`) to `/voice/command`. Despite its name it performs no speech recognition; the name is preserved so a future speech-to-text front-end can drop in without other changes.

Crazyflie control — in simulation a bash script (`fly_circle.sh`) sends `cmd_vel` Twist messages for altitude and circular flight. On hardware, `cflib MotionCommander` scripts fly three patterns at 0.33–0.5 m altitude and 0.5 m/s: circle (0.8 m radius), figure-eight (0.6 m radius), and square (0.7 m × 0.7 m). A manual keyboard script supports directional movement and emergency landing.

2.3 ROS 2 Node Graph





2.4 Dependencies & GitHub

Dependencies: `rclpy`, `std_msgs`, `geometry_msgs`, `sensor_msgs`, `irobot_create_msgs`, `ros_ign_bridge`, `cflib`, `numpy`

GitHub: For Gazebo Simulation: https://github.com/SujieZong/catch-me-if-you-can/tree/simulation For Real World: https://github.com/SujieZong/catch-me-if-you-can/tree/real_world

---

3. Experimental Design & Results

3.1 Experimental Setup

Aspect	Executed
Exp 1 — trajectories	Circle, figure-eight, square
Exp 2 — heights	0.33 m, 0.5 m, 0.75 m, 1.0 m
Visual marker	Pink ball
Operator input	Keyboard commands

3.2 Metrics and Results

Metric 1 — Tracking Reliability: whether the system chases only the intended target and does not trigger on unrelated objects.

The system was initially built around a green ball, but the green HSV filter repeatedly triggered on background objects in the lab causing the TurtleBot to chase environmental clutter. Narrowing the thresholds reduced false positives but also reduced detection range, so the marker was switched to a pink ball. Pink is largely absent from the lab environment, and after HSV recalibration no further false triggers were observed. Making thresholds tunable as live ROS 2 parameters allowed this recalibration in minutes without rebuilding.

Metric 2 — Crazyflie Flight Stability: whether the drone maintains stable altitude and executes commanded trajectories smoothly.

All experiments used a fixed takeoff height of 0.33 m or 0.5 m with a 4 s stabilisation hold after takeoff.

Behavior	Parameters	Observed Result
Hover	0.33 m height	Stable, minimal drift (≤10 cm visually)
Circle	r = 0.6–0.8 m, v = 0.5 m/s	Smooth circular path, consistent altitude
Figure-8	r = 0.6 m, v = 0.5 m/s	Clean transitions, no instability
Square	0.7 m × 0.7 m, v = 0.5 m/s	Consistent path, minor corner overshoot
Manual control	v = 0.3 m/s, Δh = 0.1 m	Immediate response, predictable motion

The drone maintained stable altitude and executed all trajectories smoothly across trials.

Metric 3 — TurtleBot Chase Behavior: whether the robot can reliably detect, pursue, and maintain a stable following distance using vision-only control.

Behavior	Control Logic	Observed Result
Target centering	Angular velocity ∝ horizontal error (`norm_cx − 0.5`)	Smooth turning, minimal oscillation
Forward motion	Speed scaled by centering, capped at 0.4 m/s	Stable approach when target is centred
Distance regulation	`area_ratio` brake zone [0.007, 0.018]	Smooth deceleration, no abrupt stops
Lost target (short)	1 s grace period on last known bearing	No jitter or sudden state switching
Lost target (long)	Transition to SEARCH, rotate in place	Reliable re-acquisition

The TurtleBot demonstrated smooth, responsive pursuit with consistent following distance and robust handling of detection flicker under real-world conditions.

---

4. Reflection

What We Delivered vs. What We Promised

All three core behaviours were delivered on real hardware. Key changes from the milestone: the visual marker switched from green to pink to eliminate lab-environment false positives, and voice commands were replaced with keyboard input. The main shortfall was a 0.5 m detection ceiling, as the marker's pixel area falls below threshold before the proposed 1.0 m altitude is reached. The Crazyflie's Flow Deck provided sufficient position stability for all tested trajectories.

Lessons Learned

Marker choice matters as much as the detection algorithm. Switching from a green ball to a pink ball eliminated false positives more effectively than narrowing HSV thresholds, which only reduced detection

range. Choosing a visually distinctive target can be more impactful than tuning the detector.

`cflib` and `rclpy` cannot share a process. The `cflib` threading model conflicts with the `rclpy` executor, requiring the Crazyflie to run as a standalone script outside the ROS graph, preventing tight real-time integration between the two robots.

Shared hardware in a multi-team lab creates unpredictable conditions. The TurtleBot's configuration was frequently changed by other teams between sessions, and the robot often could not be brought up without debugging leftover settings. Code that worked in one session may not run in the next through no fault of the software. This is a constraint of real robotics fieldwork that simulation does not replicate

What We Would Do Differently

1. Hardware-first sensing: calibrate HSV thresholds against real photos before any simulation work.
2. Use OAK-D depth output for metric distance estimation, decoupling standoff from marker apparent size.
3. Run hardware and simulation tracks in parallel from the start considering communication architectures differ significantly.

Proposed Extension

Full voice command integration. Because `/voice/command` is a standard topic, connecting a speech-to-text front-end (e.g. Whisper) requires no changes to the rest of the pipeline. If the original proposal was completed and the system would be genuinely hands-free.

Obstacle-aware navigation. Integrating Nav2 with the TurtleBot's onboard lidar would let the robot plan around obstacles in real time, maintaining pursuit through cluttered spaces the current straight-line controller cannot handle.

Bonus: Technical Hurdles and Best Practices

Crazyflie behavior is noisier than simulation suggests. Motor vibration and battery variation cause drift and hover instability not captured in simulation, making target motion less repeatable and tracking harder.

Flight parameters require empirical tuning. Low thrust causes insufficient lift; high values cause erratic motion. 0.5 m/s at 0.33–0.5 m altitude balanced stability and controllability.

A stabilisation hold after takeoff is essential. A 3–4 s hover before executing trajectories allowed the drone to settle and significantly improved smoothness.

Relative motion commands are more robust than absolute positioning. `MotionCommander` relative primitives tolerate accumulated drift naturally; absolute positioning would require localisation not available in this setup.

The pursuit system must tolerate target motion noise. The Crazyflie's trajectory is not perfectly repeatable; the TurtleBot must remain robust to drift and irregular motion.